

AccountingBench

Authenticated Submission Portal

Final Implementation Guide — Version 2.0

Reflects complete pipeline: task contribution → benchmark run → review → public leaderboard

Stack at a glance

Auth Clerk (clerk.com)

Backend FastAPI — Python

Database PostgreSQL on Render

File storage Render disk

Hosting (all) Render.com

Payments Stripe (Phase 2)

Accounts needed

1. **GitHub** — github.com

2. **Render** — render.com

3. **Clerk** — clerk.com

4. **Stripe** — stripe.com (later)

All free to start.

1. What We Are Building

The existing public AccountingBench site remains exactly as it is — no login required. This project adds two things: a private authenticated area where users contribute new benchmark tasks, and a live database-driven public leaderboard that updates automatically when you approve contributions.

1.1 The AccountingBench pipeline — how it actually works

The benchmark is a fixed, growing dataset of accounting tasks. Each task has a question, gold answer, metadata (regulatory framework, category, answer type etc.) and is stored in the PostgreSQL database. The Python script (`benchmark_run_v3_types_KopieV6.py`) runs every task against all fixed models, scores the results using the SC/MC formula or LLM-as-a-judge (GPT-5 mini), and writes the scores back to the database.

A user contributes by submitting a NEW task to the dataset. They fill in the task fields on the upload form, upload any supporting documents, and pay. Your Python script then runs that new task through all models automatically. You review the task quality. If approved, it joins the permanent dataset and its scores are averaged into the public leaderboard.

1.2 The complete flow end-to-end

Step 1 — Public site	User visits the public AccountingBench site. Leaderboard shows live aggregate scores from the database.
Step 2 — Sign in	User clicks "Sign In". Clerk handles login. Only email addresses from your approved domain list can register.
Step 3 — Landing page	User sees their previous submissions and their statuses. Link to submit a new task.
Step 4 — Upload form	User fills in the task fields (prompt, answer type, gold answer, regulatory framework, category etc.) and uploads supporting documents (PDF/Excel). They pay via Stripe.
Step 5 — Payment confirmed	Stripe webhook confirms payment. FastAPI saves the task to the database with status = pending and triggers the Python script as a background task.
Step 6 — Script runs	<code>benchmark_run_v3_types_KopieV6.py</code> runs the new task through all models (3 trials each), consolidates answers, calls the judge for open-text tasks, and writes all scores back to the database. ~30 seconds.
Step 7 — User sees result	The upload page polls every 3 seconds. When the script finishes, the raw model scores appear on the page for the user to see privately.
Step 8 — You review	In the admin interface, you review the task quality. Approve or reject with one click.
Step 9 — Leaderboard updates	If approved, the task is marked <code>is_public = True</code> . The public leaderboard API recalculates aggregate averages across all approved tasks and the numbers update automatically.

1.3 What the upload form contains

Based on the ground-truth spreadsheet columns used by the script, the form collects:

Field	Description	Required
prompt	The full question text exactly as it will be sent to the models.	Yes
answer_type	Dropdown: single_choice / multi_choice / open_text / open_numeric / journal_entry	Yes
task_type	Dropdown: interpretation_of_law / calculation / journal_entry	Yes
gold_answer	The correct answer. For choice tasks: A, B, C etc. For open tasks: the model answer.	Yes
options	For choice tasks: the answer options as JSON {A: "...", B: "...", C: "..."} For open tasks: the model answer.	If choice
regulatory_framework	Dropdown: Austrian Tax Law / IFRS / National GAAP / Mixed etc.	Yes
applicable_regulatory_year	The year the regulation applies.	Yes
category	Dropdown: Tax / Financial Accounting / Management Accounting	Yes
subcategory	Free text subcategory.	Recommended
context	Optional background text provided to the model alongside the question.	No
attached_files	URL or reference to supporting documents.	No
notes	Any additional notes for reviewers.	No
PDF upload	Supporting document (max 20 MB).	No
Excel upload	Supporting spreadsheet (max 20 MB).	No
grading_criteria	For open-text tasks: the rubric used by the LLM judge.	If open
numeric_tolerance	For open_numeric tasks: acceptable +/- tolerance band.	If numeric
education_level	Dropdown: Professional Examinations / University Master's / Secondary Vocational	Yes

2. Final Technology Stack

What	Service	Cost	Account
Public HTML files	Render Static Site	Free	Render
Private HTML pages	Render Static Site (same)	Free	— same —
Python backend	FastAPI on Render Web Service	Free	— same —
Database	PostgreSQL on Render	Free	— same —
Authentication	Clerk (clerk.com)	Free (up to 10k users)	Clerk
File uploads	Render disk storage	Free	— same —
API keys (models)	OpenAI, Anthropic, Mistral etc.	Pay per token (yours)	Each provider
Code + auto-deploy	GitHub	Free	GitHub
Payments (Phase 2)	Stripe	Free + % per txn	Stripe

The price a user pays per submission must cover your API costs: 3 trials × all models × (input + output tokens) for every task type, PLUS 1 judge call per open-text/numeric/journal-entry task using GPT-5 mini. The settings table in the database stores this price, which you set manually once you have measured the average cost per task.

3. Folder Structure

accountingbench/	← GitHub repository root
├─ public/	← existing public site (no changes)
│ ├── index.html	
│ ├── leaderboard.html	
│ ├── dashboard.html	
│ ├── methodology.html	
│ ├── about.html	
│ ├── styles.css	
│ ├── main.js	
│ └─ data.js	← replaced by API call (Section 6)
├─ auth-pages/	← private HTML pages
│ ├── sign-in.html	
│ ├── register.html	
│ ├── landing.html	← submission history + new submission link
│ └─ upload.html	← task contribution form + result display
├─ backend/	← FastAPI Python server
│ ├── main.py	← app entry point, CORS, route registration
│ ├── auth.py	← Clerk JWT verification dependency
│ ├── submissions.py	← upload form endpoint, file handling
│ ├── admin.py	← approve/reject endpoint, domain mgmt
│ ├── leaderboard.py	← public aggregate scores API endpoint
│ ├── payments.py	← Stripe webhook handler (Phase 2)
│ ├── models.py	← SQLAlchemy database table definitions
│ ├── database.py	← DB engine and session factory
│ ├── processing/	
│ │ └─ pipeline.py	← wrapper that calls benchmark_run script
│ ├── benchmark/	
│ │ └─ benchmark_run_v3_types.py	← your existing Python script (adapted)
│ ├── uploads/	← uploaded PDF and Excel files
│ └─ requirements.txt	
└─ .env	← all secrets – never commit to GitHub

4. Database Schema

Six tables cover the complete system. All are in PostgreSQL. Your Python script connects to this database directly via SQLAlchemy.

4.1 users

id	String — the Clerk user ID (e.g. user_2abc...). Primary key.
email	String, unique. Stored for display and admin.
first_name	String.
last_name	String.
created_at	DateTime.
is_active	Boolean, default True. Set False to block a user.

4.2 allowed_domains

id	Integer, primary key.
domain	String, unique. Example: kpmg.com
added_at	DateTime.
added_by	String — your admin email.

4.3 benchmark_tasks

This table replaces the ground-truth Excel spreadsheet. The 520 existing tasks are imported here. Every new user-contributed task is added here too.

id	Integer, primary key, auto-increment.
question_id	String, unique. E.g. "q_0521". Auto-generated on insert.
prompt	Text. The full question.
answer_type	String. single_choice / multi_choice / open_text / open_numeric / journal_entry
task_type	String. interpretation_of_law / calculation / journal_entry
task_format	String. Optional.
gold_answer	Text. The correct answer.
options	JSON. Answer options for choice tasks.
grading_criteria	Text. Rubric for LLM judge on open tasks.

numeric_tolerance	Float, nullable. Tolerance band for open_numeric tasks.
acceptable_variants	Text, nullable. Alternative valid answers.
context	Text, nullable.
attached_files	Text, nullable. File references.
regulatory_framework	String.
applicable_regulatory_year	Integer.
category	String. Tax / Financial Accounting / Management Accounting
subcategory	String, nullable.
skills	String, nullable.
education_level	String.
notes	Text, nullable.
source	String. "user_submitted" or "original_dataset"
submitted_by	String, nullable. FK → users.id. Null for original 520 tasks.
is_public	Boolean, default False. Set True after you approve a user submission.
created_at	DateTime.
validated_by	String, nullable. Your admin email after review.
validation_status	String. pending / approved / rejected
pdf_path	String, nullable. Path to uploaded PDF.
excel_path	String, nullable. Path to uploaded Excel.

4.4 benchmark_runs

One row per model per task submission run. Mirrors the Runs sheet from the existing Excel pipeline.

id	Integer, primary key.
run_id	String. UUID. E.g. "run_20260415_143022_a1b2c3d4"
task_id	Integer, FK → benchmark_tasks.id
model_name	String. E.g. "gpt-5.4"
run_timestamp	DateTime.
temperature	Float. Always 0.0.
n_trials	Integer. Always 3.
system_prompt_version	String. E.g. "v3_types"

dataset_version	String. E.g. "v3"
judge_model	String. Always "gpt-5-mini"
inference_notes	Text. Any additional run metadata.

4.5 benchmark_outputs

One row per model per task. Stores all trial answers, scores, and judge output. Mirrors the Outputs sheet from the existing Excel pipeline.

id	Integer, primary key.
run_id	String, FK → benchmark_runs.run_id
task_id	Integer, FK → benchmark_tasks.id
model_name	String.
model_answer_1	Text. Trial 1 answer.
model_confidence_1	Float. Trial 1 confidence.
model_answer_2	Text. Trial 2 answer.
model_confidence_2	Float.
model_answer_3	Text. Trial 3 answer.
model_confidence_3	Float.
final_answer	Text. Consolidated answer (majority vote or judge consolidation).
avg_model_confidence	Float.
score_percent_sc_mc	Float, nullable. SC/MC formula score (choice tasks only).
judge_score_percent	Float, nullable. LLM-as-a-judge score (open tasks only).
judge_confidence	Float, nullable.
final_score_percent	Float. The score that counts for the leaderboard.
evaluation_method	String. sc_mc_formula or judge.
token_input	Integer.
token_output	Integer.
token_reasoning	Integer, nullable.
evaluated_at_utc	DateTime.
evaluation_notes	Text.

4.6 submissions

Tracks each user's contribution from form submission through payment, script execution, and review.

id	Integer, primary key.
user_id	String, FK → users.id
task_id	Integer, FK → benchmark_tasks.id. Set after task is inserted.
status	String. pending → processing → done → error
payment_status	String. unpaid → paid → refunded. (Phase 2)
stripe_payment_id	String, nullable. Stripe Payment Intent ID. (Phase 2)
price_charged	Integer, nullable. Amount in cents. (Phase 2)
submitted_at	DateTime.
completed_at	DateTime, nullable. When the script finishes.

4.7 settings

Single-row configuration table. Change the price here without touching code.

id	Integer. Always 1.
price_per_submission	Integer. Cents. E.g. 5000 = €50.00. Set after measuring average API cost. (Phase 2)
currency	String. E.g. eur. (Phase 2)
updated_at	DateTime.

5. Adapting the Python Script for the Database

The existing `benchmark_run_v3_types_KopieV6.py` reads from Excel files and writes to Excel files. It needs to be adapted to read tasks from PostgreSQL and write results back to PostgreSQL. The core logic — the three-trial inference loop, SC/MC scoring, judge calls, consolidation — stays completely unchanged.

5.1 What changes in the script

Currently (Excel)	Adapted (Database)
<code>pd.read_excel(GROUND_TRUTH_XLSX)</code> — reads all tasks	<code>db.query(BenchmarkTask).filter_by(id=task_id)</code> — reads one specific new task
<code>pd.read_excel(MODEL_CALL_XLSX)</code> — reads model call template	Task row is passed directly as a Python dict — no Excel needed
<code>append_row_respecting_formulas(ws_runs, ...)</code> — writes to Excel	<code>db.add(BenchmarkRun(...)) + db.commit()</code> — writes to PostgreSQL
<code>append_row_respecting_formulas(ws_out, ...)</code> — writes to Excel	<code>db.add(BenchmarkOutput(...)) + db.commit()</code> — writes to PostgreSQL
<code>wb.save(EVALUATION_XLSX)</code> — saves the workbook	Changes are committed row by row as the script runs
Script is run from the command line manually	Script is triggered as a FastAPI background task after form submission

5.2 The pipeline wrapper (`backend/processing/pipeline.py`)

Rather than modifying the script heavily, wrap it. The wrapper handles the database connection and calls the adapted script functions:

```
from backend.database import SessionLocal
from backend.models import BenchmarkTask, BenchmarkRun, BenchmarkOutput, Submission
from backend.benchmark.benchmark_run_v3_types import run_single_task

def run_pipeline(submission_id: int):
    db = SessionLocal()
    try:
        submission = db.query(Submission).filter_by(id=submission_id).first()
        submission.status = "processing"
        db.commit()

        task = db.query(BenchmarkTask).filter_by(id=submission.task_id).first()

        # Run the task through all models
        for model_name in ALL_MODELS:
            results = run_single_task(task, model_name, db)
            # run_single_task does: 3 trials → consolidate → judge → save to DB
```

```
        submission.status = "done"
        submission.completed_at = datetime.utcnow()
        db.commit()

    except Exception as e:
        submission.status = "error"
        db.commit()
    finally:
        db.close()
```

5.3 ALL_MODELS list

The fixed list of models the script runs against every new task. Add or remove models here when the benchmark expands:

```
ALL_MODELS = [
    "gpt-5.4",
    "gpt-5.2",
    "claude-opus-4-6",
    "claude-sonnet-4-6",
    "gpt-5-mini",
    "Mistral-Large-3",
    "grok-4-fast-reasoning",
    "gpt-4o",
    "DeepSeek-V3.2-2",
    "mercury-2",
]
```

5.4 API keys in .env

All model provider API keys live in the .env file on the server. The script reads them via os.environ exactly as it does now:

```
OPENAI_API_KEY=sk-...
MODEL_REGISTRY_JSON={"claude-opus-4-6": {"provider": "anthropic", ...}, ...}
# Add keys for Mistral, xAI, DeepSeek, Inception Labs here
# Never commit .env to GitHub
```

6. Live Public Leaderboard — Replacing data.js

Currently leaderboard numbers are hardcoded in data.js. After this project, the public site fetches live aggregate scores from the database via a single API endpoint. Only approved tasks (`is_public = True`) are included in the calculation.

6.1 The leaderboard API endpoint (backend/leaderboard.py)

This endpoint calculates the average `final_score_percent` per model across all approved tasks:

```
@app.get("/api/leaderboard")
async def get_leaderboard():
    """
    Returns average scores per model across all is_public=True tasks.
    Groups by model_name, averages final_score_percent.
    Also breaks down by category (Tax, Fin. Acc., Mgmt. Acc.)
    """
    db = SessionLocal()

    # Get all approved task IDs
    approved = db.query(BenchmarkTask.id).filter_by(is_public=True).all()
    approved_ids = [t.id for t in approved]

    # Query outputs for approved tasks only
    outputs = db.query(BenchmarkOutput).filter(
        BenchmarkOutput.task_id.in_(approved_ids)
    ).all()

    # Aggregate per model
    from collections import defaultdict
    scores = defaultdict(list)
    by_category = defaultdict(lambda: defaultdict(list))

    for o in outputs:
        task = db.query(BenchmarkTask).get(o.task_id)
        scores[o.model_name].append(o.final_score_percent)
        by_category[o.model_name][task.category].append(o.final_score_percent)

    result = []
    for model, sc in scores.items():
        result.append({
            "model": model,
            "overall": round(sum(sc)/len(sc), 1),
            "financial": round(avg(by_category[model]["Financial Accounting"]), 1),
            "management": round(avg(by_category[model]["Management Accounting"]), 1),
            "tax": round(avg(by_category[model]["Tax"]), 1),
            "n_tasks": len(set(o.task_id for o in outputs if o.model_name==model)),
        })
    return sorted(result, key=lambda x: -x["overall"])
```

6.2 Updating the public HTML pages

In the public HTML pages (index.html, leaderboard.html, dashboard.html), replace the hardcoded MODELS and lbData constants in data.js with a fetch() call:

```
// In main.js — replace hardcoded data with live fetch
async function loadLeaderboard() {
  const res = await fetch("https://your-api.onrender.com/api/leaderboard");
  const data = await res.json();
  // data is the same shape as the old lbData array
  renderLeaderboard(data);
  updateHeroStats(data);
}
window.addEventListener("load", loadLeaderboard);
```

The shape of the API response matches the existing lbData array format used in main.js and data.js, so renderLeaderboard(), drawChart(), and all existing chart functions work without changes. Only the source of the data changes — from a hardcoded file to a live API call.

6.3 Caching the leaderboard response

The leaderboard does not change often — only when you approve a submission. Add a simple 5-minute cache to avoid hitting the database on every public page load:

```
from fastapi_cache import FastAPICache
from fastapi_cache.decorator import cache

@app.get("/api/leaderboard")
@cache(expire=300) # cache for 5 minutes
async def get_leaderboard():
  ...
```

7. Admin Review Interface

After the Python script finishes, you see the new task in a simple admin page. You review the task text for quality and click Approve or Reject. No code-level access needed for the day-to-day review.

7.1 Admin page (auth-pages/admin.html)

A protected HTML page only accessible with your admin email (checked by Clerk + FastAPI). It shows a list of all tasks with status = pending:

- Task prompt, answer type, category, regulatory framework, gold answer.
- Uploaded documents as download links.
- The model scores from the benchmark run (so you can see what the script produced).
- Approve and Reject buttons.

7.2 Approve / Reject endpoints (backend/admin.py)

```
@app.post("/admin/tasks/{task_id}/approve")
async def approve_task(task_id: int, user=Depends(require_admin)):
    db = SessionLocal()
    task = db.query(BenchmarkTask).filter_by(id=task_id).first()
    task.is_public = True
    task.validation_status = "approved"
    task.validated_by = user
    db.commit()
    # Leaderboard cache auto-expires within 5 minutes
    return {"ok": True}

@app.post("/admin/tasks/{task_id}/reject")
async def reject_task(task_id: int, user=Depends(require_admin)):
    db = SessionLocal()
    task = db.query(BenchmarkTask).filter_by(id=task_id).first()
    task.is_public = False
    task.validation_status = "rejected"
    task.validated_by = user
    db.commit()
    return {"ok": True}
```

7.3 What the user sees after rejection

Nothing is shown to the user about the review decision. On their landing page, the submission appears with status "rejected" and no score is shown. The task is not included in any public numbers. No explanation is sent.

8. Step-by-Step Implementation

Phase 1 — Project Setup + Database

~1.5 hours

1. Create the folder structure from Section 3 and push to GitHub.
2. Create a PostgreSQL database on Render (New → PostgreSQL, free tier).
3. Set up the Python virtual environment and install dependencies:

```
pip install fastapi uvicorn sqlalchemy alembic psycpg2-binary
pip install python-dotenv aiofiles python-multipart clerk-backend-api
pip install pandas openpyxl pypdf requests openai anthropic
pip install fastapi-cache2 stripe
```

4. Write models.py with all six tables from Section 4.
5. Run Alembic to create the tables:

```
alembic init migrations
alembic revision --autogenerate -m "initial schema"
alembic upgrade head
```

6. Import the 520 existing tasks into benchmark_tasks. Write a one-time migration script that reads your existing matrikelnummer_ground_truth_template.xlsx and inserts all rows with is_public=True and source="original_dataset".
7. Verify in TablePlus or DBeaver: 520 rows in benchmark_tasks, all with is_public=True.

Test: the 520 existing tasks are in the database. The /api/leaderboard endpoint returns the same numbers as data.js currently shows (because all 520 are approved). This confirms the database schema is correct before building anything else.

Phase 2 — Authentication (Clerk)

~1 hour

8. Create a Clerk application. Under Restrictions → Allowlist, enable domain-based registration.
9. Add allowed domains (e.g. kpmg.com) in the Clerk dashboard. Add/remove any time — no code change needed.
10. Add the Clerk publishable key and secret key to .env.
11. Write backend/auth.py — the get_current_user dependency that verifies the Clerk JWT from the session cookie (~10 lines).
12. Write backend/auth.py — a require_admin dependency that additionally checks the user email matches ADMIN_EMAIL from .env.
13. Add the "Sign In" link to the public nav bar in each HTML page:

```
<a href="/auth-pages/sign-in.html" class="btn-primary"
  style="font-size:11px;padding:8px 18px;">Sign In</a>
```

14. Mount the Clerk sign-in and register components in sign-in.html and register.html.

Reference: clerk.com/docs/quickstarts/javascript

Test: register with an allowed email domain. Verify the email. Sign in. Confirm redirect to landing.html. Try registering with a non-allowed domain — confirm it is blocked.

Phase 3 — Upload Form + Python Script Integration

~3 hours

3a — Upload HTML page (auth-pages/upload.html)

15. Build the form with all fields from the table in Section 1.3. Use the existing styles.css — .meth-card for the form container, existing input styles.
16. Use dropdown menus (HTML <select>) for answer_type, task_type, category, regulatory_framework, and education_level — these must match the exact values the Python script expects.
17. Add two file inputs: one accepting .pdf, one accepting .xlsx/.xls.
18. Show/hide optional fields based on answer_type selection (e.g. show options only when single_choice or multi_choice is selected, show grading_criteria only for open tasks).

3b — Submission endpoint (backend/submissions.py)

19. Create POST /submissions/prepare. It must:
 - Require authentication via Depends(get_current_user).
 - Validate all required fields are present.
 - Validate file types (extension + MIME type).
 - Insert the task into benchmark_tasks with validation_status = "pending", is_public = False.
 - Insert a row into submissions with status = "pending".
 - Save uploaded files to uploads/{user_id}/{task_id}/.
 - Trigger run_pipeline(submission_id) as a BackgroundTask.
 - Return {"submission_id": id, "task_id": task_id}.

3c — Adapt the Python script

20. Copy benchmark_run_v3_types_KopieV6.py to backend/benchmark/benchmark_run_v3_types.py.
21. Extract the per-task logic into a run_single_task(task_dict, model_name, db) function that accepts a task as a Python dict (instead of reading from Excel) and writes results to the database (instead of Excel).
22. The three-trial loop, call_llm_json(), call_judge(), score_sc_mc_percent(), majority_vote(), and all scoring logic stay exactly the same.
23. Write backend/processing/pipeline.py as the wrapper shown in Section 5.2.

3d — Status polling and result display (upload.html)

24. After the form submits and returns a `submission_id`, show a progress indicator.

25. Poll GET `/submissions/{id}/status` every 3 seconds:

```
const poll = setInterval(async () => {
  const res = await fetch(`/submissions/${id}/status`);
  const data = await res.json();
  if (data.status === "done") {
    clearInterval(poll);
    displayResults(data.model_scores);
  }
  if (data.status === "error") {
    clearInterval(poll);
    showError();
  }
}, 3000);
```

26. The `/submissions/{id}/status` endpoint returns:

```
{
  "status": "done",
  "model_scores": [
    {"model": "gpt-5.4", "score": 72.0, "method": "sc_mc_formula"},
    {"model": "gpt-5.2", "score": 68.0, "method": "sc_mc_formula"},
    // ... all 10 models
  ]
}
```

Test: submit a new task. Confirm the task appears in `benchmark_tasks`. Confirm 10 run rows appear in `benchmark_runs` and 10 output rows in `benchmark_outputs` after ~30 seconds. Confirm the status changes to "done" and the scores appear on the page.

Phase 4 — Landing Page + Admin Interface

~1.5 hours

4a — Landing page (auth-pages/landing.html)

27. Protect the page with a Clerk session check at the top.

28. Fetch and display the user's submission history from GET `/submissions/mine`. Show: task question (first 80 chars), submitted date, status badge, and for done submissions a "View Result" link.

29. Add a prominent "Contribute New Task" button linking to `upload.html`.

4b — Admin page (auth-pages/admin.html)

30. Protect with `require_admin`. If the logged-in user is not the admin email, redirect to `landing.html`.

31. Fetch pending tasks from GET `/admin/tasks?status=pending`.

32. Display each task with: full prompt, answer type, category, gold answer, model scores from the benchmark run, and document download links.
33. Approve and Reject buttons POST to `/admin/tasks/{id}/approve` and `/admin/tasks/{id}/reject`.
34. Add a domain management section: list current allowed domains, form to add a new one, delete button for each.

Phase 5 — Live Leaderboard + Public Site Update

~1 hour

35. Write the GET `/api/leaderboard` endpoint from Section 6.1.
36. Add `fastapi-cache2` and wrap the endpoint with `@cache(expire=300)`.
37. In `main.js`, replace the hardcoded `MODELS` and `lbData` with a `fetch()` call to `/api/leaderboard` as shown in Section 6.2.
38. Test: approve one of the user-contributed tasks via the admin page. Reload the public leaderboard within 5 minutes. Confirm the numbers update to reflect the new approved task.
39. Verify the dashboard charts, radar chart, and heatmap all still render correctly — they use the same data shape.

Phase 6 — Deployment to Render

~1 hour

40. Push everything to GitHub.
41. On Render: New → Static Site → connect GitHub → publish directory: public → deploy. (Public site)
42. On Render: New → Static Site → connect GitHub → publish directory: auth-pages → deploy. (Private pages)
43. On Render: New → Web Service → connect GitHub → root directory: backend → start command:

```
uvicorn main:app --host 0.0.0.0 --port 10000
```
44. In Render Web Service → Environment Variables, add every `.env` variable including all API keys.
45. Update the `DATABASE_URL` to the Render PostgreSQL external URL.
46. In `main.py`, add your two Render Static Site URLs to CORS allowed origins.
47. In Clerk dashboard → Domains, add both Render Static Site URLs.
48. Update all `fetch()` URLs in HTML pages from `localhost:8000` to the Render Web Service URL.
49. Run `alembic upgrade head` against the production database to create the tables.
50. Run the one-time migration script to import the 520 tasks into production.

Important: Render's free Web Service sleeps after 15 minutes of inactivity. The first request after sleep takes ~30 seconds to wake. For a benchmark portal used sporadically this is acceptable. The \$7/month paid tier removes this. Budget for this before launch.

Phase 7 — Payments with Stripe (Phase 2 — build later)

Design is in place — add when ready

The database already has `payment_status`, `stripe_payment_id`, `price_charged` in submissions, and the settings table with `price_per_submission`. No schema changes needed.

Steps when ready

51. Measure the average API cost per submitted task by looking at `token_input` + `token_output` across several submissions and multiplying by each provider's price per token. Set `price_per_submission` in the settings table to cover that cost plus your margin.
52. Install Stripe: `pip install stripe`.
53. On form submit, create a Stripe Payment Intent for `price_per_submission` cents and redirect the user to Stripe Checkout.
54. Add POST `/payments/webhook`. Verify the Stripe-Signature header. On `payment_intent.succeeded`, set `payment_status` = "paid" and trigger `run_pipeline()` as a `BackgroundTask`.
55. Without payment, the script does not run. With payment confirmed by Stripe, it runs.

Reference: stripe.com/docs/checkout/quickstart

9. Security Checklist

Auth	Clerk handles all passwords, tokens, and email verification. You never store or see passwords.
JWT verification	Every protected FastAPI endpoint calls <code>get_current_user</code> which verifies the Clerk token. Never trust user ID from the frontend.
Admin endpoints	<code>/admin/*</code> routes call <code>require_admin</code> which checks the verified user email against <code>ADMIN_EMAIL</code> in <code>.env</code> .
API keys	All model provider API keys in <code>.env</code> only. Never in code. Never in GitHub. Added via Render Environment Variables.
File validation	Check extension AND MIME type for uploads. Reject anything not PDF or Excel. Set 20 MB max file size.
SQL injection	SQLAlchemy ORM prevents this by default. Never build raw SQL strings with user input.
CORS	Only list your Render Static Site URLs in <code>allow_origins</code> . Never use <code>allow_origins=["*"]</code> in production.
HTTPS	Render serves HTTPS on all services automatically.
Stripe webhooks	Always verify the Stripe-Signature header before processing any payment event.
Submission access	<code>/submissions/mine</code> filters by the verified user ID. Users cannot see each other's tasks or scores.
Leaderboard data	GET <code>/api/leaderboard</code> filters by <code>is_public=True</code> . Pending and rejected tasks are never exposed publicly.

10. Effort Estimate

Phase	Task	Time
1	Setup, database, import 520 tasks	1.5 hrs
2	Clerk auth, domain restriction, sign-in pages	1 hr
3	Upload form, submission endpoint, Python script adaptation, polling	3 hrs
4	Landing page, admin review interface	1.5 hrs
5	Live leaderboard API, update public site	1 hr
6	Deployment to Render	1 hr
—	Total Phase 1 (without payments)	~9 hours
7	Stripe pay-per-submission (Phase 2)	2–3 hrs

Build and test locally through Phase 5 before deploying. FastAPI's automatic docs at <http://localhost:8000/docs> let you test every endpoint without any frontend. Run the Python script locally first against a test task to confirm it connects to the database correctly — this is the most complex integration point.

11. All Key Links

Accounts

GitHub	github.com — free
Render	render.com — free tier, no credit card
Clerk	clerk.com — free up to 10,000 monthly active users
Stripe	stripe.com — free account; pay per transaction when live (Phase 2)
OpenAI	platform.openai.com — for GPT models and the judge (GPT-5 mini)
Anthropic	console.anthropic.com — for Claude models
Mistral	console.mistral.ai — for Mistral-Large-3
xAI	console.x.ai — for grok-4-fast
DeepSeek	platform.deepseek.com — for DeepSeek-V3.2-2
Inception	inceptionlabs.ai — for mercury-2

Documentation

FastAPI — getting started	fastapi.tiangolo.com/tutorial/
FastAPI — background tasks	fastapi.tiangolo.com/tutorial/background-tasks/
FastAPI — file uploads	fastapi.tiangolo.com/tutorial/request-files/
FastAPI — CORS	fastapi.tiangolo.com/tutorial/cors/
SQLAlchemy — ORM quickstart	docs.sqlalchemy.org/en/20/orm/quickstart.html
Alembic — migration tutorial	alembic.sqlalchemy.org/en/latest/tutorial.html
fastapi-cache2	github.com/long2ice/fastapi-cache
Clerk — JavaScript quickstart	clerk.com/docs/quickstarts/javascript
Clerk — FastAPI integration	clerk.com/docs/backend-requests/overview
Clerk — Allowlist	clerk.com/docs/authentication/allowlist
Render — deploy FastAPI	render.com/docs/deploy-fastapi
Render — PostgreSQL	render.com/docs/postgresql-creating-connecting
Stripe — Checkout quickstart	stripe.com/docs/checkout/quickstart
Stripe — Webhooks	stripe.com/docs/webhooks
TablePlus (free DB viewer)	tableplus.com
DBeaver (free DB viewer)	dbeaver.io

